

Методические указания к лабораторной работе

Тема: «Отличия языков программирования C и C++. Основные возможности C++»

1. Цель работы

Изучение ключевых отличий C++ от C, приобретение практических навыков использования возможностей современного C++:

- работа с потоками ввода-вывода (`std::cin` / `std::cout`);
 - инициализация переменных с помощью унифицированного синтаксиса `{}`;
 - использование констант (`const`, `constexpr`);
 - ссылки и передача параметров по ссылке;
 - логический тип `bool`;
 - значения параметров по умолчанию;
 - управление динамической памятью через умные указатели (`std::unique_ptr`, `std::shared_ptr`);
 - работа со строками (`std::string`);
 - использование стандартных контейнеров (`std::vector`, `std::list`, `std::map`);
 - применение алгоритмов стандартной библиотеки (`std::sort`, `std::find` и др.);
 - файловый ввод-вывод и форматирование вывода.
-

2. Теоретическая часть и примеры

2.1. Комментарии

В C++ доступны два стиля комментариев: многострочные (`/* */`) и однострочные (`//`).

Пример:

```
#include <iostream>

int main() {
    // Это однострочный комментарий
    std::cout << "Hello, World!\n";    /* вывод на экран */
    return 0;
}
```

2.2. Пространства имён

Пространства имён позволяют избегать конфликтов имён. Вся стандартная библиотека находится в пространстве `std`.

Пример:

```
#include <iostream>

namespace MySpace {
    void print(int x) {
        std::cout << "MySpace: " << x << '\n';
    }
}

int main() {
    MySpace::print(42);           // обращение через квалификатор
    using namespace MySpace;     // теперь print доступна без префикса
    print(100);
    return 0;
}
```

2.3. Инициализация переменных

В C++ рекомендуется использовать унифицированную инициализацию фигурными скобками (`{}`). Она предотвращает сужающие преобразования.

Пример:

```
#include <iostream>

int main() {
    int a {10};                  // прямая списочная инициализация
    double b {3.14};
    int c {};                    // value-инициализация (обнуление)
    // int d {3.14};            // ошибка: сужающее преобразование

    std::cout << a << ' ' << b << ' ' << c << '\n';
    return 0;
}
```

2.4. Константы и constexpr

- `const` – значение не может быть изменено после инициализации (может вычисляться во время выполнения).
- `constexpr` – значение известно на этапе компиляции.

Пример:

```
#include <iostream>

int main() {
    const int daysInWeek {7};
    constexpr double pi {3.1415926535898};

    int userAge {};
```

```

std::cin >> userAge;
const int constAge {userAge}; // OK, время выполнения

// constexpr int bad {userAge}; // ошибка: не константа компиляции

std::cout << daysInWeek << ' ' << pi << ' ' << constAge << '\n';
return 0;
}

```

2.5. Ссылки

Ссылка – альтернативное имя для уже существующего объекта. Обязательно инициализируется при создании и не может быть перенаправлена. Часто используется для передачи параметров в функцию без копирования.

Пример:

```

#include <iostream>

void swap(int& x, int& y) {
    int tmp = x;
    x = y;
    y = tmp;
}

int main() {
    int a {5}, b {10};
    std::cout << "До: a=" << a << ", b=" << b << '\n';
    swap(a, b);
    std::cout << "После: a=" << a << ", b=" << b << '\n';

    int& ref {a}; // ref ссылается на a
    ref = 100;    // теперь a = 100
    std::cout << "a=" << a << '\n';
    return 0;
}

```

2.6. Логический тип bool

Переменные типа `bool` принимают значения `true` или `false`. Совместимы с целыми числами: `true` → 1, `false` → 0.

Пример:

```

#include <iostream>

int main() {
    bool flag {true};
    std::cout << "flag = " << flag << '\n'; // выведет 1
    std::cout << std::boolalpha << flag << '\n'; // выведет true
}

```

```

    int x {10};
    bool isPositive {x > 0};
    std::cout << std::boolalpha << isPositive << '\n';
    return 0;
}

```

2.7. Параметры функций по умолчанию

Можно задать значения по умолчанию для последних параметров функции.

Пример:

```

#include <iostream>

void printLine(char fill = '-', int length = 10) {
    for (int i = 0; i < length; ++i)
        std::cout << fill;
    std::cout << '\n';
}

int main() {
    printLine();           // -----
    printLine('*');        // *****
    printLine('=', 5);     // =====
    return 0;
}

```

2.8. Умные указатели (вместо new/delete)

- `std::unique_ptr` – единоличное владение.
- `std::shared_ptr` – разделяемое владение (счётчик ссылок).
- Создаются через `std::make_unique` и `std::make_shared`.

Пример:

```

#include <iostream>
#include <memory>

struct Point {
    double x, y;
    Point(double x_, double y_) : x(x_), y(y_) {}
    void print() const { std::cout << '(' << x << ',' << y << ")\n"; }
};

int main() {
    auto p1 = std::make_unique<Point>(1.0, 2.5);
    p1->print();

    std::shared_ptr<int> sp = std::make_shared<int>(100);
}

```

```
std::cout << *sp << '\n';

// память освободится автоматически при выходе из области видимости
return 0;
}
```

2.9. Строки std::string

Класс `std::string` предоставляет безопасную работу со строками переменной длины.

Пример:

```
#include <iostream>
#include <string>

int main() {
    std::string name;
    std::cout << "Введите полное имя: ";
    std::getline(std::cin >> std::ws, name);    // чтение с игнорированием ведущих
    пробелов

    std::string greeting {"Привет, "};
    greeting += name;
    greeting += '!';

    std::cout << greeting << '\n';
    std::cout << "Длина строки: " << greeting.length() << '\n';
    return 0;
}
```

2.10. Контейнеры STL

2.10.1. std::vector

Динамический массив.

Пример:

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> numbers {10, 5, 8, 3, 12};
    numbers.push_back(7);    // добавляем в конец

    std::sort(numbers.begin(), numbers.end());

    std::cout << "Отсортированный вектор: ";
    for (int n : numbers)
```

```
    std::cout << n << ' ';
    std::cout << '\n';

    std::cout << "Размер: " << numbers.size() << '\n';
    return 0;
}
```

2.10.2. std::list

Двусвязный список. Эффективная вставка/удаление в середине.

Пример:

```
#include <iostream>
#include <list>
#include <algorithm>

int main() {
    std::list<double> lst {1.1, 2.2, 3.3, 4.4};
    lst.push_front(0.0);
    lst.push_back(5.5);

    lst.remove(3.3);                // удалить все элементы со значением 3.3

    for (double d : lst)
        std::cout << d << ' ';
    std::cout << '\n';
    return 0;
}
```

2.10.3. std::map

Ассоциативный массив (словарь). Хранит пары ключ-значение.

Пример:

```
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> ages;
    ages["Alice"] = 30;
    ages["Bob"] = 25;

    ages.insert({"Charlie", 35});

    for (const auto& [name, age] : ages)
        std::cout << name << ": " << age << '\n';
}
```

```
    auto it = ages.find("Alice");
    if (it != ages.end())
        std::cout << "Возраст Алисы: " << it->second << '\n';
    return 0;
}
```

2.11. Алгоритмы STL

Большое количество готовых алгоритмов в заголовке `<algorithm>`.

Пример:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>    // для accumulate

int main() {
    std::vector<int> v {1, 2, 3, 4, 5};

    // сумма элементов
    int sum = std::accumulate(v.begin(), v.end(), 0);
    std::cout << "Сумма: " << sum << '\n';

    // найти первый чётный элемент
    auto it = std::find_if(v.begin(), v.end(), [](int x){ return x % 2 == 0; });
    if (it != v.end())
        std::cout << "Первый чётный: " << *it << '\n';

    // применить функцию ко всем элементам
    std::for_each(v.begin(), v.end(), [](int& x){ x *= 2; });
    std::cout << "Удвоенные: ";
    for (int x : v) std::cout << x << ' ';
    std::cout << '\n';
    return 0;
}
```

2.12. Файловый ввод-вывод

Потоки `ifstream` (чтение), `ofstream` (запись), `fstream` (чтение/запись).

Пример записи в файл:

```
#include <iostream>
#include <fstream>
#include <vector>

int main() {
    std::ofstream outFile("data.txt");
    if (!outFile) {
        std::cerr << "Ошибка открытия файла!\n";
    }
}
```

```
        return 1;
    }

    std::vector<int> data {10, 20, 30, 40};
    for (size_t i = 0; i < data.size(); ++i) {
        outFile << data[i];
        if (i != data.size() - 1)
            outFile << ' ';
    }
    outFile << '\n';
    outFile.close();
    return 0;
}
```

Пример чтения из файла:

```
#include <iostream>
#include <fstream>
#include <vector>

int main() {
    std::ifstream inFile("data.txt");
    if (!inFile) {
        std::cerr << "Файл не найден!\n";
        return 1;
    }

    std::vector<int> data;
    int value;
    while (inFile >> value) {
        data.push_back(value);
    }
    inFile.close();

    std::cout << "Прочитано " << data.size() << " чисел: ";
    for (int v : data) std::cout << v << ' ';
    std::cout << '\n';
    return 0;
}
```

2.13. Форматированный вывод

Манипуляторы из `<iomanip>` позволяют управлять шириной поля, точностью, системой счисления и т.д.

Пример:

```
#include <iostream>
#include <iomanip>

int main() {
```

```
double pi = 3.1415926535;
int number = 255;

std::cout << "Обычный вывод: " << pi << '\n';
std::cout << std::fixed << std::setprecision(3);
std::cout << "С 3 знаками: " << pi << '\n';

std::cout << "\nШестнадцатеричное 255: " << std::hex << number << '\n';
std::cout << std::dec << std::showbase;
std::cout << "С основанием: " << std::hex << number << '\n';

std::cout << std::dec << std::noshowbase;
std::cout << "Таблица умножения:\n";
for (int i = 1; i <= 5; ++i) {
    for (int j = 1; j <= 5; ++j) {
        std::cout << std::setw(4) << i * j;
    }
    std::cout << '\n';
}
return 0;
}
```

3. Задание

Напишите программу на C++:

- Создайте в git отдельный проект
- В проект добавьте преподавателя (логин vstolchnev) с ролью разработчик или администратор
- Создайте отдельную ветку git (lab3)
- Добавьте в проект файл .gitignore для языка c++
- Напишите программу согласно варианту
- Загрузите программу в git в текщей ветке (должна быть lab3)
- В gitverse создайте запрос на слияние в ветку master
- Добавьте преподавателя в качестве reviewer

Дополнительные условия:

1. **Вводит данные** согласно вашему варианту:
 - либо с клавиатуры (по запросу),
 - либо из текстового файла (имя файла может задаваться пользователем).
2. **Сохраняет данные** в один или несколько стандартных контейнеров (`std::vector`, `std::list`, `std::map` и т.п.) – выбор контейнера также указан в варианте.
3. **Выполняет обработку** в соответствии с вариантом:
 - вычисление статистических характеристик,
 - фильтрацию,
 - поиск,
 - преобразование,
 - сортировку,
 - и т.д.

4. **Использует умные указатели** для управления динамической памятью, если в задании требуется создание объектов в динамической области (например, для хранения данных, не помещающихся в стеке). В большинстве случаев достаточно стандартных контейнеров, но при необходимости динамического массива следует применять `std::unique_ptr<T[]>` или `std::shared_ptr<T[]>`.
5. **При объявлении любых переменных обязательно применяет унифицированную инициализацию `{}`** (например, `int count{}`; вместо `int count = 0;`).
6. **Выводит результаты:**
 - на экран в отформатированном виде (с использованием манипуляторов `<iomanip>`),
 - при необходимости также в файл (имя файла указывается в варианте).
7. **Комментирует код**, поясняя ключевые моменты.

Все вспомогательные функции должны использовать передачу параметров по ссылке там, где это уместно, и константные ссылки для неизменяемых объектов.

4. Варианты заданий

Вариант 1. Анализ последовательности чисел

- **Ввод:** с клавиатуры целое число N, затем N вещественных чисел.
- **Контейнер:** `std::vector<double>`.
- **Обработка:** вычислить среднее арифметическое, минимальное и максимальное значение. Найти количество чисел, превышающих среднее.
- **Вывод:** числа в исходном порядке, затем статистика, отформатированная с точностью 2 знака после запятой.
- **Пояснение:** используйте алгоритмы `std::accumulate`, `std::minmax_element`, `std::count_if`.

Вариант 2. Частотный словарь слов

- **Ввод:** из текстового файла `input.txt` произвольный текст (слова разделены пробелами, знаками препинания).
- **Контейнер:** `std::map<std::string, int>` для подсчёта частоты слов.
- **Обработка:** привести все слова к нижнему регистру, удалить знаки препинания. Вывести таблицу: слово и количество вхождений, отсортированную по убыванию частоты.
- **Вывод:** на экран в виде двух колонок, выровненных по левому краю (ширина 15 для слова, ширина 5 для числа).
- **Пояснение:** используйте `std::ifstream`, `std::getline`, `std::stringstream`, алгоритмы преобразования.

Вариант 3. Сортировка списка студентов

- **Ввод:** с клавиатуры количество студентов, затем для каждого – фамилия (строка) и оценка (целое).
- **Контейнер:** `std::list<std::pair<std::string, int>>`.
- **Обработка:** отсортировать список по убыванию оценки, при равных оценках – по алфавиту.
- **Вывод:** таблица с заголовком, фамилия выровнена по левому краю (ширина 20), оценка по правому (ширина 3).
- **Пояснение:** используйте метод `list::sort` с пользовательским компаратором.

Вариант 4. Матрица и её транспонирование

- **Ввод:** из файла `matrix.txt` сначала два числа М и N (строки и столбцы), затем М*N целых чисел.
- **Контейнер:** `std::vector<std::vector<int>>>` для хранения матрицы.
- **Обработка:** вычислить транспонированную матрицу. Найти суммы элементов каждой строки и каждого столбца исходной матрицы.
- **Вывод:** исходная матрица, транспонированная матрица, суммы строк и столбцов. Все числа выводить с шириной поля 5.
- **Пояснение:** работа с вложенными векторами, использование `push_back`.

Вариант 5. Умные указатели и массив точек

- **Ввод:** с клавиатуры N (количество точек), затем координаты x, y для каждой точки.
- **Контейнер:** `std::vector<std::shared_ptr<std::pair<double, double>>>`.
- **Обработка:** вычислить расстояние от каждой точки до начала координат. Найти точку с максимальным расстоянием.
- **Вывод:** список точек (x, y) и расстояния (точность 3), в конце – информация о самой удалённой точке.
- **Пояснение:** используйте `std::make_shared`, алгоритмы `std::max_element` с лямбда-функцией.

Вариант 6. Палиндромы в списке строк

- **Ввод:** с клавиатуры последовательность строк (пока не встретится пустая строка).
- **Контейнер:** `std::list<std::string>`.
- **Обработка:** оставить только те строки, которые являются палиндромами (читаются одинаково слева направо и справа налево, без учёта регистра). Отсортировать их по длине.
- **Вывод:** исходные строки, затем отобранные палиндромы.
- **Пояснение:** функция проверки палиндрома с использованием `std::equal`, игнорирование регистра через `tolower`.

Вариант 7. Статистика роста студентов

- **Ввод:** из файла `heights.txt` список роста (целые сантиметры) – по одному числу в строке.
- **Контейнер:** `std::vector<int>`.
- **Обработка:** вычислить средний рост, медиану, моду (наиболее часто встречающееся значение). Если мод несколько – вывести все.
- **Вывод:** исходные данные, среднее (с одним знаком), медиана, мода(ы). Для вывода использовать формат с выравниванием.
- **Пояснение:** для моды можно использовать `std::map<int, int>`.

Вариант 8. Обработка заказов

- **Ввод:** с клавиатуры число заказов, затем для каждого – название товара (строка) и количество (целое).
- **Контейнер:** `std::map<std::string, int>` – суммарное количество по каждому товару.
- **Обработка:** объединить заказы с одинаковыми товарами. Вывести товары, отсортированные по алфавиту.
- **Вывод:** таблица: товар, общее количество. Использовать выравнивание колонок.
- **Пояснение:** при вводе сразу заполнять `map`.

Вариант 9. Числа Фибоначчи и контейнер

- **Ввод:** целое число K .
- **Контейнер:** `std::vector<unsigned long long>` для хранения первых K чисел Фибоначчи.
- **Обработка:** заполнить вектор числами Фибоначчи ($F_1=1, F_2=1$). Затем вычислить сумму чётных чисел среди них.
- **Вывод:** первые K чисел (по 10 в строке, ширина 10), затем сумму чётных.
- **Пояснение:** динамическое выделение памяти не требуется, вектор сам управляет памятью.

Вариант 10. Стек и очередь

- **Ввод:** с клавиатуры последовательность целых чисел (оканчивается 0).
- **Контейнеры:** `std::stack<int>` (на базе `std::deque`) и `std::queue<int>`.
- **Обработка:** поместить все числа в стек и в очередь. Затем извлечь из стека и очереди по одному элементу и сравнить их. Вывести, равны ли они.
- **Вывод:** исходная последовательность, результат сравнения первых элементов.
- **Пояснение:** адаптеры `stack` и `queue`, методы `push`, `pop`, `top`, `front`.

Вариант 11. Множество уникальных чисел

- **Ввод:** из файла `numbers.txt` произвольное количество целых чисел (разделённых пробелами).
- **Контейнер:** `std::set<int>`.
- **Обработка:** вставить числа в множество (автоматически останутся только уникальные). Найти минимальный и максимальный элементы.
- **Вывод:** уникальные числа в порядке возрастания, затем минимум и максимум.
- **Пояснение:** множество отсортировано, можно использовать `*set.begin()` и `*set.rbegin()`.

Вариант 12. Список с удалением дубликатов

- **Ввод:** с клавиатуры N , затем N строк.
- **Контейнер:** `std::list<std::string>`.
- **Обработка:** удалить все дубликаты (оставить только первые вхождения). Отсортировать оставшиеся строки по длине, а при равной длине – лексикографически.
- **Вывод:** исходный список, список после удаления дубликатов, отсортированный список.
- **Пояснение:** для удаления дубликатов можно использовать вспомогательный `std::set`.

Вариант 13. Вектор указателей на объекты

- **Ввод:** с клавиатуры количество кругов, затем для каждого – радиус (`double`).
- **Контейнер:** `std::vector<std::unique_ptr<class Circle>>`, где `Circle` – структура с полем `radius` и методом `area()`.
- **Обработка:** вычислить площадь каждого круга. Найти круг с максимальной площадью.
- **Вывод:** радиусы и площади (точность 2), а также данные круга с максимальной площадью.
- **Пояснение:** `Circle` – простая структура, не содержащая виртуальных функций. Использовать `std::make_unique<Circle>(radius)`.

Вариант 14. Трансформация списка

- **Ввод:** из файла `data.txt` вещественные числа.

- **Контейнер:** `std::list<double>`.
- **Обработка:** применить к каждому числу функцию $f(x) = x^2 + 2x + 1$. Удалить все числа, которые после преобразования стали отрицательными.
- **Вывод:** исходные числа, преобразованные числа (после удаления), каждое с тремя знаками.
- **Пояснение:** использовать `std::transform` и `std::remove_if` совместно с методом `erase`.

Вариант 15. Частота символов

- **Ввод:** строка текста (может содержать пробелы) с клавиатуры.
- **Контейнер:** `std::map<char, int>` для подсчёта частоты символов (буквы приводить к нижнему регистру, игнорировать не-буквы).
- **Обработка:** вывести символы и их частоту в порядке убывания частоты.
- **Вывод:** таблица: символ (в кавычках) и количество.
- **Пояснение:** для сортировки по значению придётся скопировать пары в вектор и отсортировать.

Вариант 16. Работа с датами

- **Ввод:** с клавиатуры число N, затем N дат в формате `день месяц год` (целые числа).
- **Контейнер:** `std::vector<std::tuple<int,int,int>>`.
- **Обработка:** отсортировать даты по возрастанию (сначала год, потом месяц, потом день). Найти самую раннюю и самую позднюю дату.
- **Вывод:** исходные даты, отсортированные, ранняя и поздняя.
- **Пояснение:** использовать `std::sort` с лямбда-компаратором.

Вариант 17. Очередь с приоритетом

- **Ввод:** с клавиатуры последовательность целых чисел (пока не введено 0).
- **Контейнер:** `std::priority_queue<int>`.
- **Обработка:** поместить числа в очередь с приоритетом (по умолчанию максимальный элемент на вершине). Извлечь все элементы и вывести их в порядке убывания.
- **Вывод:** исходные числа, затем числа в порядке убывания.
- **Пояснение:** адаптер `priority_queue`, методы `push`, `pop`, `top`.

Вариант 18. Полином

- **Ввод:** степень полинома M и его коэффициенты (double), затем значение x.
- **Контейнер:** `std::vector<double>` для коэффициентов (от младшей степени к старшей).
- **Обработка:** вычислить значение полинома в точке x по схеме Горнера.
- **Вывод:** коэффициенты полинома, значение в точке x с точностью 4.
- **Пояснение:** схема Горнера реализуется циклом.

Вариант 19. Анализ оценок группы

- **Ввод:** из файла `grades.txt` сначала количество студентов, затем для каждого – фамилия и оценка (целое).
- **Контейнер:** `std::map<std::string, int>` (фамилия – оценка).
- **Обработка:** найти средний балл, количество отличников (оценка 5), количество неуспевающих (оценка 2). Вывести фамилии всех, у кого оценка выше среднего.
- **Вывод:** статистика и список фамилий, выровненных по левому краю.

- **Пояснение:** использовать `std::accumulate` для среднего.

Вариант 20. Сортировка слиянием двух списков

- **Ввод:** с клавиатуры размеры двух списков и их элементы (целые).
- **Контейнеры:** два `std::list<int>`.
- **Обработка:** объединить два отсортированных списка в один отсортированный (не используя глобальную сортировку, а метод `merge`).
- **Вывод:** исходные списки, объединённый список.
- **Пояснение:** предварительно отсортировать каждый список методом `sort()`.

Вариант 21. Двумерный массив как вектор векторов

- **Ввод:** из файла `table.txt` числа в виде таблицы: сначала M и N, затем M строк по N целых чисел.
- **Контейнер:** `std::vector<std::vector<int>>`.
- **Обработка:** вычислить произведение всех положительных элементов, количество отрицательных.
- **Вывод:** исходная матрица, произведение, количество отрицательных.
- **Пояснение:** обход вложенными циклами.

Вариант 22. Поиск подстроки

- **Ввод:** две строки (первая – текст, вторая – подстрока).
- **Контейнер:** `std::string` (основная строка).
- **Обработка:** найти все вхождения подстроки в текст (с учётом регистра) и вывести позиции (индексы начала). Использовать `std::string::find`.
- **Вывод:** исходный текст, подстрока, список позиций.
- **Пояснение:** цикл с обновлением стартовой позиции.

Вариант 23. Умный указатель на массив

- **Ввод:** с клавиатуры N, затем N целых чисел.
- **Контейнер:** `std::unique_ptr<int[]>` для хранения массива (не используя `std::vector`).
- **Обработка:** вычислить сумму и среднее арифметическое.
- **Вывод:** исходные числа, сумма, среднее (с двумя знаками).
- **Пояснение:** `auto arr = std::make_unique<int[]>(N);`, затем работа как с обычным массивом.

Вариант 24. Шифр Цезаря

- **Ввод:** строка текста и целое число сдвига (ключ).
- **Контейнер:** `std::string`.
- **Обработка:** зашифровать строку, сдвигая каждую букву латинского алфавита на ключ (остальные символы оставить без изменений). Реализовать функцию шифрования, принимающую строку по ссылке.
- **Вывод:** исходная строка, зашифрованная строка.
- **Пояснение:** проверка диапазонов `'a'-'z'`, `'A'-'Z'`.

Вариант 25. Статистика покупок

- **Ввод:** из файла `purchases.txt` строки вида: `товар цена количество` (товар – строка без пробелов, цена – `double`, количество – `int`).
 - **Контейнер:** `std::map<std::string, double>` для общей стоимости по каждому товару (цена * количество).
 - **Обработка:** суммировать стоимость по каждому товару. Найти товар с максимальной общей стоимостью.
 - **Вывод:** таблица товаров и общей стоимости (с двумя знаками), в конце – лидер.
 - **Пояснение:** использование `std::ifstream`, разбор строки через `std::stringstream`.
-

5. Контрольные вопросы

1. В чём отличие однострочных и многострочных комментариев в C++?
2. Для чего нужны пространства имён? Приведите пример.
3. Почему не рекомендуется использовать директиву `using namespace std;` в глобальной области видимости?
4. Какие существуют способы инициализации переменных в C++? Какой способ предпочтителен и почему?
5. Что произойдёт при попытке сужающего преобразования в списочной инициализации?
6. В чём разница между `const` и `constexpr`? Приведите примеры.
7. Что такое ссылка? В чём её отличие от указателя?
8. Можно ли изменить ссылку так, чтобы она ссылалась на другой объект? Почему?
9. Как передать параметр в функцию, чтобы избежать копирования, но гарантировать, что функция не изменит аргумент?
10. Какие значения может принимать тип `bool`? Как он связан с целыми числами?
11. Для чего используются параметры функций по умолчанию? Какие ограничения на их расположение в списке параметров?
12. Чем `std::unique_ptr` отличается от `std::shared_ptr`?
13. Как создать умный указатель с помощью `std::make_unique` и `std::make_shared`? Преимущества перед `new`?
14. Какие методы класса `std::string` вы знаете для работы с подстроками?
15. В чём разница между `std::vector` и `std::list`? Когда что использовать?
16. Как добавить элемент в конец вектора? Как вставить в середину?
17. Какие способы обхода контейнеров существуют в C++ (индексы, итераторы, range-based for)?
18. Что такое итератор? Какие виды итераторов бывают?
19. Как отсортировать вектор с помощью STL?
20. Как найти элемент в контейнере по условию? Приведите пример с `std::find_if`.
21. Что такое лямбда-функция? Для чего она удобна при работе с алгоритмами?
22. Как открыть файл для чтения в C++? Какие режимы открытия существуют?
23. Как проверить, успешно ли открылся файл?
24. Как прочитать из файла строку целиком, включая пробелы?
25. Как управлять форматированием вывода: ширина поля, точность, система счисления?
26. Что делает манипулятор `std::ws`?
27. Чем отличается `std::cin >>` от `std::getline`?
28. Для чего нужен `std::endl`? Чем он отличается от `'\n'`?
29. Какие алгоритмы из `<numeric>` вы знаете?
30. Как работает `std::accumulate`? Приведите пример с суммой и с произведением.

31. Как удалить элемент из списка по значению? (метод `remove`)
 32. В чём разница между `map::operator[]` и `map::insert`?
 33. Можно ли хранить в `std::set` пользовательские типы? Что для этого нужно?
 34. Как использовать умные указатели с массивами (`std::unique_ptr<T[]>`)?
 35. Что произойдёт при попытке разыменования нулевого указателя?
 36. Какие методы для работы с размером есть у контейнеров?
 37. Что такое адаптеры контейнеров (`stack`, `queue`, `priority_queue`)?
 38. Как получить доступ к первому элементу очереди?
 39. Чем отличается `push` от `emplace`?
 40. Какие существуют способы инициализации `std::pair`?
-

6. Приложение: справочник по используемым конструкциям и методам

6.1. Ввод-вывод (`<iostream>`, `<iomanip>`)

- **`std::cin`** – объект стандартного ввода.
 - `std::cin >> var` – извлечение данных в переменную `var`.
 - `std::cin >> std::ws` – игнорирование ведущих пробелов.
- **`std::cout`** – объект стандартного вывода.
 - `std::cout << expr` – вывод выражения `expr`.
 - `std::endl` – вывод перевода строки и сброс буфера.
- **`std::getline(std::cin, str)`** – чтение строки с пробелами до разделителя (по умолчанию `'\n'`).
- **Манипуляторы** (требуют `<iomanip>`):
 - `std::setw(n)` – установка ширины поля вывода в `n` символов.
 - `std::setprecision(n)` – установка точности (количество знаков после запятой для `fixed/scientific`).
 - `std::fixed` – фиксированная запись вещественных чисел.
 - `std::scientific` – экспоненциальная запись.
 - `std::boolalpha` / `std::noboolalpha` – вывод `bool` как `"true"/"false"` или `1/0`.
 - `std::showbase` / `std::noshowbase` – показывать основание системы счисления.
 - `std::uppercase` / `std::nouppercase` – буквы в шестнадцатеричных числах.
 - `std::dec`, `std::hex`, `std::oct` – системы счисления.
 - `std::left`, `std::right`, `std::internal` – выравнивание.
 - `std::setfill(c)` – символ заполнитель.

6.2. Строки (`<string>`)

- **Конструкторы:**
 - `std::string s;` – пустая строка.
 - `std::string s("hello");` – из литерала.
 - `std::string s(10, '*');` – из 10 символов `'*'`.
- **Методы:**
 - `size()`, `length()` – длина строки.
 - `empty()` – проверка на пустоту.
 - `clear()` – очистка.
 - `c_str()` – возвращает `const char*` (для совместимости с C).

- `data()` – возвращает указатель на данные (с C++11 аналогично `c_str()`).
- `append(str)` – добавление строки.
- `push_back(ch)` – добавление символа.
- `pop_back()` – удаление последнего символа.
- `insert(pos, str)` – вставка.
- `erase(pos, count)` – удаление.
- `replace(pos, count, str)` – замена.
- `substr(pos, count)` – извлечение подстроки.
- `find(str, pos)` – поиск первого вхождения.
- `rfind(str, pos)` – поиск последнего вхождения.
- `compare(str)` – сравнение (возвращает <0, 0, >0).
- `operator+, +=` – конкатенация.
- `operator==, !=, <, <=, >, >=` – сравнение.

6.3. Контейнеры

6.3.1. `std::vector<T>`

- **Конструкторы:** `vector()`, `vector(n)`, `vector(n, val)`, `vector(init_list)`.
- **Методы:**
 - `push_back(const T& val)` – добавление в конец.
 - `pop_back()` – удаление последнего.
 - `size()` – количество элементов.
 - `empty()` – проверка на пустоту.
 - `clear()` – очистка.
 - `resize(n)` – изменение размера.
 - `reserve(n)` – резервирование памяти.
 - `operator[](size_t i)` – доступ без проверки границ.
 - `at(size_t i)` – доступ с проверкой (бросает исключение).
 - `front()`, `back()` – первый и последний элемент.
 - `begin()`, `end()` – итераторы.
 - `insert(iterator pos, const T& val)` – вставка.
 - `erase(iterator pos) / erase(first, last)` – удаление.
 - `data()` – указатель на массив.

6.3.2. `std::list<T>`

- **Конструкторы:** аналогично `vector`.
- **Методы:**
 - `push_front()`, `pop_front()` – вставка/удаление в начале.
 - `push_back()`, `pop_back()` – в конце.
 - `size()`, `empty()`, `clear()`.
 - `front()`, `back()`.
 - `begin()`, `end()`.
 - `insert()`, `erase()`.
 - `remove(const T& val)` – удаление всех элементов, равных `val`.
 - `remove_if(Predicate pred)` – удаление по условию.

- `sort()` – сортировка (по умолчанию `<`).
- `unique()` – удаление последовательных дубликатов.
- `merge(list& other)` – слияние двух отсортированных списков.
- `splice(iterator pos, list& other)` – перемещение элементов из другого списка.

6.3.3. `std::map<Key, T>`

- **Конструкторы:** `map()`, `map(init_list)`.
- **Методы:**
 - `operator[](const Key& k)` – доступ по ключу (создаёт элемент, если отсутствует).
 - `insert({key, value})` – вставка пары.
 - `emplace(key, value)` – вставка с конструированием на месте.
 - `find(const Key& k)` – поиск, возвращает итератор или `end()`.
 - `count(const Key& k)` – количество элементов с ключом (0 или 1).
 - `size()`, `empty()`, `clear()`.
 - `begin()`, `end()` – итераторы, указывающие на пары `std::pair<const Key, T>`.
 - `erase(iterator)` / `erase(key)` – удаление.

6.4. Алгоритмы (`<algorithm>`, `<numeric>`)

- `std::sort(first, last)` – сортировка диапазона (random-access итераторы).
- `std::stable_sort(first, last)` – устойчивая сортировка.
- `std::find(first, last, value)` – поиск значения.
- `std::find_if(first, last, pred)` – поиск по условию.
- `std::count(first, last, value)` – подсчёт вхождений.
- `std::count_if(first, last, pred)` – подсчёт по условию.
- `std::for_each(first, last, f)` – применение функции к каждому элементу.
- `std::transform(first, last, result, op)` – преобразование элементов.
- `std::accumulate(first, last, init)` – свёртка (сумма, произведение и т.д.).
- `std::min_element`, `std::max_element` – поиск минимума/максимума.
- `std::minmax_element` – поиск минимума и максимума одновременно.
- `std::remove(first, last, value)` – перемещает элементы, не равные `value`, в начало; возвращает новый конец.
- `std::remove_if(first, last, pred)` – аналогично по условию.
- `std::copy(first, last, out)` – копирование.
- `std::unique(first, last)` – удаление последовательных дубликатов.

6.5. Умные указатели (`<memory>`)

- `std::unique_ptr<T>` – уникальное владение.
 - Создание: `auto p = std::make_unique<T>(args);`
 - Для массивов: `auto p = std::make_unique<T[]>(size);`
 - Методы: `get()` – получить сырой указатель, `reset()` – освободить владение, `swap()`.
- `std::shared_ptr<T>` – разделяемое владение.
 - Создание: `auto p = std::make_shared<T>(args);`
 - Методы: `use_count()` – количество владельцев, остальные аналогично `unique_ptr`.
- `std::weak_ptr<T>` – слабая ссылка (не увеличивает счётчик).

6.6. Файловые потоки (<fstream>)

- **std::ifstream** – входной файловый поток.
 - **open(filename, mode)** – открыть файл (режимы: **std::ios::in** – чтение).
 - **is_open()** – проверка открытия.
 - **close()** – закрыть.
 - Оператор **>>** – чтение форматированных данных.
 - **getline(stream, str)** – чтение строки.
 - **eof()** – проверка конца файла.
 - **fail()**, **bad()**, **good()** – проверка состояния.
- **std::ofstream** – выходной файловый поток.
 - Режимы: **std::ios::out** (запись, перезапись), **std::ios::app** (добавление в конец).
 - Оператор **<<** – вывод.
- **std::fstream** – файловый поток для чтения и записи.
 - Режимы: комбинация **in | out**, **in | out | app** и т.д.

6.7. Адаптеры контейнеров

- **std::stack<T>** – стек (по умолчанию на базе **deque**).
 - **push()**, **pop()**, **top()**, **empty()**, **size()**.
- **std::queue<T>** – очередь (по умолчанию на базе **deque**).
 - **push()**, **pop()**, **front()**, **back()**, **empty()**, **size()**.
- **std::priority_queue<T>** – очередь с приоритетом (на базе **vector**).
 - **push()**, **pop()**, **top()**, **empty()**, **size()**.

6.8. Прочие полезные объекты

- **std::pair<T1, T2>** – пара значений.
 - Доступ к полям: **first**, **second**.
 - Создание: **std::make_pair(a, b)**, или **{a, b}**.
- **std::tuple<Args...>** – кортеж.
 - Доступ: **std::get<i>(tuple)**.

Примечание. В работе разрешается использовать любые описанные выше средства, а также при необходимости дополнительные заголовочные файлы (<cmath>, <ctime> и т.п.). Все программы должны быть написаны в соответствии с современным стандартом C++ (C++17/20). Использование **new/delete** напрямую не допускается – только через умные указатели.